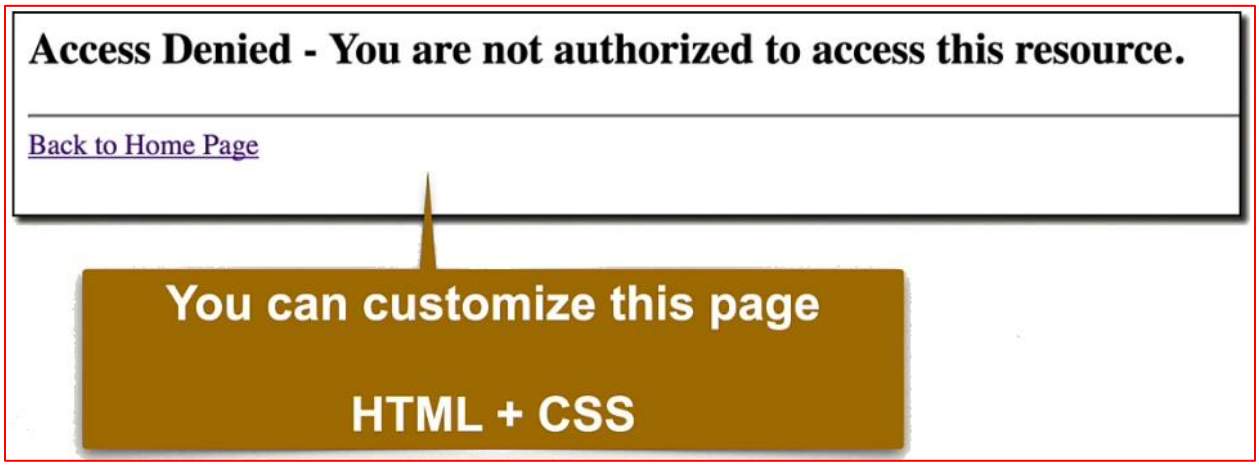
Spring MVC Security - Custom Access Denied Page

In this lesson, we are going to explore how to set up a custom "Access Denied" page. You've likely seen the default "403 Forbidden" page. It can be quite alarming to the average user of your application. It's a message that might scare them away.



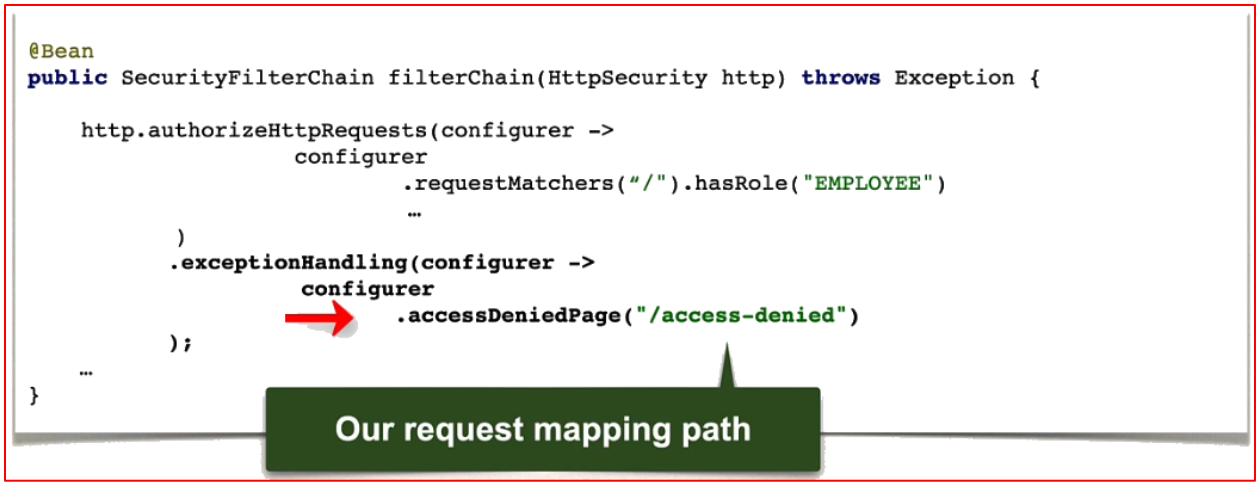
Our goal here is to provide a custom page that conveys the same information without alarming the user. We will craft our own custom error message, allowing us to control both the appearance and the wording of the page. You can freely customize this page with your own HTML, CSS, and even add in some stylish bootstrap elements.



Development Process

Step One: Configuring the Custom Page for Access Denied

The first step involves configuring the custom page for access denied situations. Within the code, you'll use the method `exceptionHandling().accessDeniedPage`, and then provide the request mapping path. This path will be used by Spring Security to display the custom "Access Denied" page whenever there's an authorization error or the user attempts to access a forbidden page.



Step Two: Creating the Supporting Controller Code and View Page

Now, let's move on to step two: creating the supporting controller code and view page. This process is standard Spring MVC, so you might already be familiar with it.

We will cover these steps in the next lesson because you've seen the controller and view page coding before, so we'll save those details for later.

Configuring Custom Page for Access Denied

We'll now open up `DemoSecurityConfig.java`. Here, we'll use `.exceptionHandling().accessDeniedPage`, and set our request mapping path to `access-denied`. You can name this anything you'd like.



Creating Supporting Controller Code and View Pages

Next, let's create the supporting controller code and view pages. We'll open up `LoginController.java`, where we usually place my access denied code. You could actually place the request mapping anywhere, but grouping security-related stuff in one controller is a good practice.

After stretching out the code, we'll add a quick comment, and then copy a similar section of code and modify it. For the `GetMapping`, we'll have `access-denied`, and then update the method name. Finally, we'll set the return to `access-denied`.

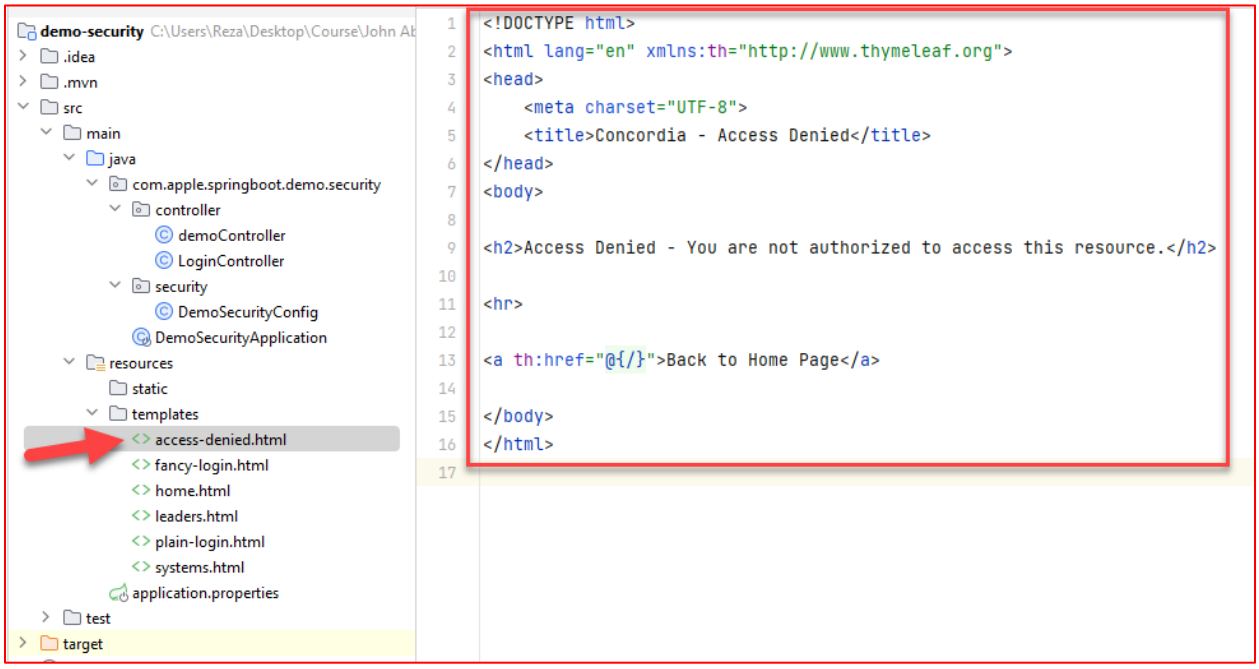


Since we mentioned `access-denied`, we need to create this page.

Creating a New File for Access Denied

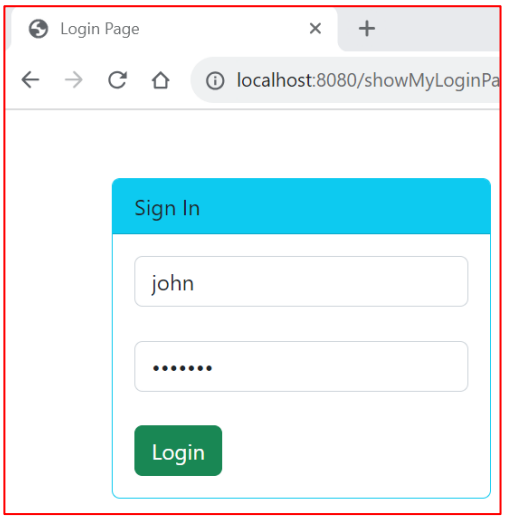
We'll start by creating a new file for our custom "Access Denied" page. We'll drop in a quick title like "Concordia - Access Denied". The body of the page can display any error message we want, such as "*Access Denied, you are not authorized to access this page.*" We have full control over the content.

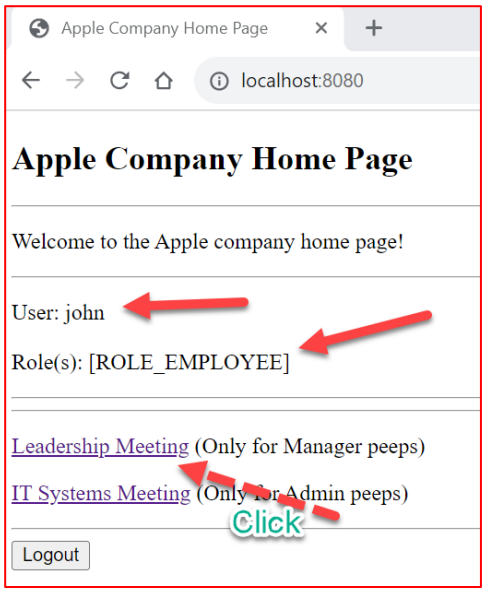
One best practice is to always provide a link back to the homepage. It offers the user a path back to the main site. With that, our custom page is complete, plain and simple.

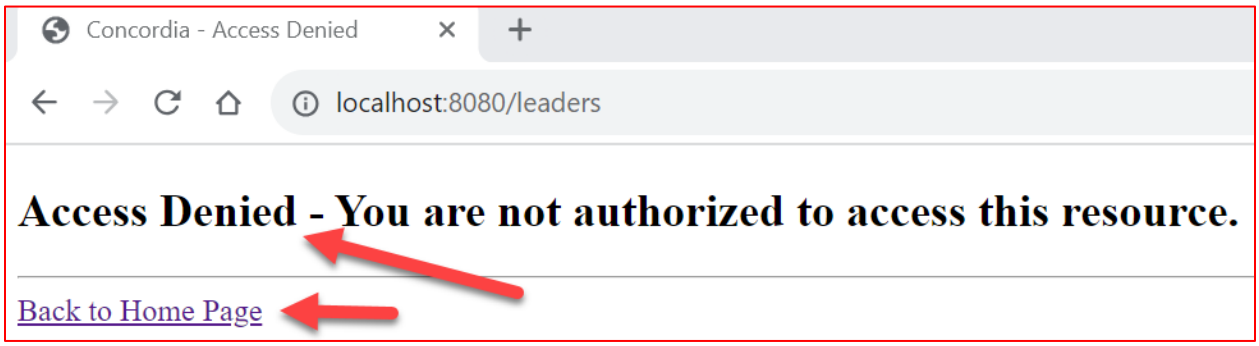


Testing the Custom Page

Now, let's test our new custom page. We'll run the application and log in with user credentials. If a user with limited roles tries to access forbidden links, our custom "Access Denied" page will be displayed.







As you can see, this custom page is more user-friendly and less alarming than the default "403 Forbidden" message. It's working, and we're in good shape!